iND83213A CS01 SDK Release Note
V1.12

1 REVISION HISTORY

Table 1 Revision History

Rev #	Date	SDV Version and Action	By
1.00	10/07/2023	Ver: libdev_iND83213A_CS01_App_B0.01.01_lin_2.0.8_colorlib_2.2.2 (1)Make adc clock to be 4MHz	Winter
1.01	10/11/2023	Ver: libdev_iND83213A_CS01_App_B0.01.02_lin_2.0.8_colorlib_2.2.2 (1) Add ADC Data Filter Function for negative value	Winter
1.02	10/20/2023	Ver: libdev_iND83213A_CS01_App_B0.01.03_lin_2.0.8_colorlib_2.2.2 (1) ADC negative processing before calling adcCallback	Winter
1.03	10/20/2023	Ver: libdev_iND83213A_CS01_App_B0.01.04_lin_2.0.8_colorlib_2.2.2 (1) update lin default params for SNPD	Winter
1.04	12/19/2023	Ver: libdev_iND83213A_CS01_App_B0.01.05_lin_2.0.8_colorlib_2.2.2 (1) Porting linslaveTask_Private.c	Winter
1.0.5	3/21/2024	Ver: libdev_iND83213A_CS01_App_B0.01.06_lin_2.0.8_colorlib_2.2.2 (1) Update LIN Performance Params for LIN_AA (2) Fix sleep issue @Private_linsavetask mode	Winter
1.0.6	4/2/2024	Ver: libdev_iND83213A_CS01_App_B0.01.07_lin_2.0.9_colorlib_2.2.2 (1) Update LIN Performance Params for LIN_AA (2) Fix pwm device IRQ API (3) lin tack lib update J2602 Error Report logic @J2602 Mode	Winter
1.0.7	4/19/2024	Ver: libdev_iND83213A_CS01_App_B0.01.08_lin_2.0.9_colorlib_2.2.2 (1) Resolve the issue of abnormal sleep mode branches	Levi Ye
1.0.8	7/26/2024	Ver: libdev_iND83213A_CS01_App_B0.01.09_lin_2.1.0_colorlib_2.2.2 (1) Fix the issue of report led error when it is turned off (2) Add an linstack api for Once Force 3D response without 3C require (3) Add special functional NAD (0x7E) handler	Levi Ye
1.09	9/29/2024	Ver: libdev_iND83213A_CS01_App_B0.01.10_lin_2.1.0_colorlib_2.2.2 (1) Update void PMU_EnterDeepSleepMode(void) API for reduce sleep current	Winter

Rev #	Date	SDV Version and Action	By
1.10	12/6/2024	Ver: libdev_iND83213A_CS01_App_B0.01.11_lin_2.1.0_colorlib_2.2.2 (1) Fix the issue of Buck Set Freq Api Inaccurate settings	Levi
1.11	01/02/2025	Ver: libdev_iND83213A_CS01_App_B0.01.12_lin_2.1.0_colorlib_2.2.2 (1) Update Temperature Calculation Method (2) Add Handling Negative Voltage	Levi
1.12	02/28/2025	Ver: libdev_iND83213A_CS01_App_B0.01.13_lin_2.1.0_colorlib_2.2.2 (1) Fix the issue of abnormal wake-up. (2) Fix the issue of led false fault report by two-point calibration (3) Fix the issue of system clock division mismatch	Levi

2 TABLE OF CONTENTS

1 REVISION HISTORY 2
2 TABLE OF CONTENTS 4
3 V1.12 6
3.1 ChangeVersion 6
3.2 Bug Fix 6
3.2.1 Fix the issue of abnormal wake-up. 6
3.2.2 Fix the issue of led false fault report by two-point calibration 7
3.2.3 Fix the issue of system clock division mismatch 8
4 V1.11 10
4.1 ChangeVersion 10
4.2 Bug Fix 10
4.2.1 Update Temperature Calculation Method 10
4.2.2 Add Handling Negative Voltage 11
5 V1.10 12
5.1 ChangeVersion 12
5.2 Bug Fix 12
5.2.1 Fix the issue of Buck Set Freq Api Inaccurate settings 12
6 V1.09 13
6.1 Version Change 13
6.2 Feature Update 13
6.2.1 Update PMU_EnterDeepSleepMode(void) API 13
7 V1.08 14
7.1 Version Change 14
7.2 Bug Fix 14
7.2.1 Fix the issue of reporting led error when it is turned off 14
7.3 NEW Feature 15
7.3.1 Add an linstack api for Once Force 3D response without 3C require. 15
7.3.2 Add special functional NAD (0x7E) handler. 16
8 V1.07 17
8.1 Version Change 17
8.2 BUG Fix 17
8.2.1 Resolve the issue of abnormal sleep mode branches 17
9 V1.06 18
9.1 ChangeVersion 18
9.2 LIN stack Update 18
9.2.1 Update error report logic @J2602 Mode 18
9.3 BUG FIXED 19

9.3.1 Update LIN device Params for improve LIN_AA performance 19
9.3.2 Fix pwm device driver API error 20

10 V1.05 21

10.1 ChangeVersion 21

10.2 BUG FIXED 21

10.2.1 Update LIN device Params for improve LIN_AA performance 21
10.2.2 Fix sleep issue @Private_linsavetask mode 22

11 V1.04 23

11.1 ChangeVersion 23

11.2 BUG FIXED 23

11.2.1 Proting linslaveTask_Private.c 23

12 V1.03 24

12.1 ChangeVersion 24

12.2 Default Status Modify 24

12.2.1 Default Params Modify For SNPD 24

13 V1.02 25

13.1 ChangeVersion 25

13.2 BUG Fixed 25

13.2.1 Modify code position 25

14 V1.01 26

14.1 ChangeVersion 26

14.2 BUG Fixed 26

14.2.1 None 26

14.3 New Feature 27

14.3.1 Add ADC Code Filter 27

15 V1.00 28

15.1 ChangeVersion 28

15.2 BUG Fixed 28

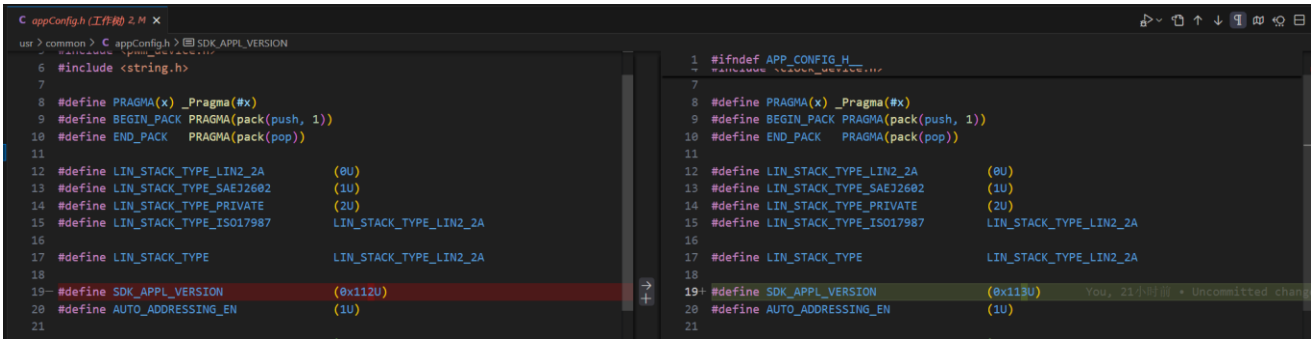
15.2.1 Make ADC clock source to be 4MHz 28

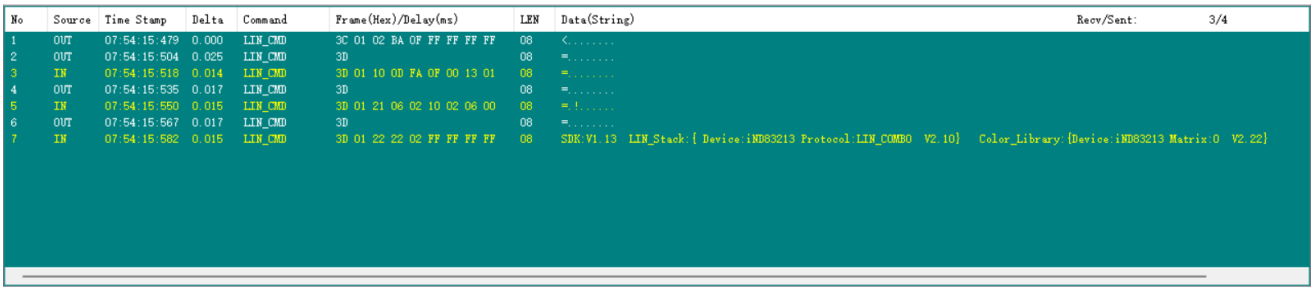
3 V1.12

3.1 CHANGEVERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.12_lin_2.1.0_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.13_lin_2.1.0_colorlib_2.2.2





3.2 BUG FIX

3.2.1 Fix the issue of abnormal wake-up.

3.2.1.1 issue description:

When using sleep mode and designating LINS as the wake-up source, it is necessary to disable LINM wake-up enable. Before executing LINM_WUP_ENA = 0, be sure to clear LINM_Sleep.

3.2.1.2 solution:

Make sure linm sleep = 0 when entering sleepmode：

```c
void PMU_EnterDeepSleepMode(void)
{
    TRIMHV_SFRS->CFG_ACCESS.CFG_ACCESS_KEY    = 0x0BU;
    CRGA_SFRS->MODULERST.WORD = (MODUE_SOFT_RESET_PWM | MODUE_SOFT_RESET_ADC);
    /* Add GPIO function here for low power mode*/

    /*  End of GPIO settigns   */
    TRIMHV_SFRS->OVTEMP_CONFIG.TEMPSENSE_EN     = 0U;     /* disbale temperature sensor when sleep */
    TRIMHV_SFRS->VBATTRIM.BAT_UV_EN             = 0U;
    TRIMHV_SFRS->VBATTRIM.BAT_OV_EN             = 0U;

    TRIMHV_SFRS->LINS.LINS_RX_BIAS_BOOST        = 0U;
    TRIMHV_SFRS->LINM.LINM_RX_BIAS_BOOST        = 0U;

    /*******Do not modify PD1V5_ENA_HIBERNATE********/
    TRIMHV_SFRS->PMU_CTRL.PD1V5_ENA_HIBERNATE   = 0U;

    WICA_SFRS->CTRL.LINM_IRQCLR         = 1U;
    WICA_SFRS->CTRL.LINM_IRQENA         = 0U;
    TRIMHV_SFRS->LIN_WUP.LINM_WUP_ENA   = 0U;
    NVIC_ClearPendingIRQ(WULINM_IRQn);

    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.SLEEP = 1U;)  */
    WICA_SFRS->CTRL.LINS_IRQCLR         = 1U;
    WICA_SFRS->CTRL.LINS_IRQENA         = 1U;
    TRIMHV_SFRS->LIN_WUP.LINS_WUP_ENA   = 1U;
    NVIC_EnableIRQ(WULINS_IRQn);
```

```c
void PMU_EnterDeepSleepMode(void)
{
    TRIMHV_SFRS->CFG_ACCESS.CFG_ACCESS_KEY    = 0x0BU;
    CRGA_SFRS->MODULERST.WORD = (MODUE_SOFT_RESET_PWM | MODUE_SOFT_RESET_ADC);
    /* Add GPIO function here for low power mode*/

    /*  End of GPIO settigns   */
    TRIMHV_SFRS->OVTEMP_CONFIG.TEMPSENSE_EN     = 0U;     /* disbale temperature sensor when sleep */
    TRIMHV_SFRS->VBATTRIM.BAT_UV_EN             = 0U;
    TRIMHV_SFRS->VBATTRIM.BAT_OV_EN             = 0U;

    TRIMHV_SFRS->LINS.LINS_RX_BIAS_BOOST        = 0U;
    TRIMHV_SFRS->LINM.LINM_RX_BIAS_BOOST        = 0U;

    /*******Do not modify PD1V5_ENA_HIBERNATE********/
    TRIMHV_SFRS->PMU_CTRL.PD1V5_ENA_HIBERNATE   = 0U;

    /* Disable linm wakeup detect and IRQ  */
    LINM_SFRS->CTRL.SLEEP = 0U;
    WICA_SFRS->CTRL.LINM_IRQCLR         = 1U;
    WICA_SFRS->CTRL.LINM_IRQENA         = 0U;
    TRIMHV_SFRS->LIN_WUP.LINM_WUP_ENA   = 0U;
    NVIC_ClearPendingIRQ(WULINM_IRQn);

    /* Enable linm wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.SLEEP = 1U;)  */
    WICA_SFRS->CTRL.LINS_IRQCLR         = 1U;
    WICA_SFRS->CTRL.LINS_IRQENA         = 1U;
    TRIMHV_SFRS->LIN_WUP.LINS_WUP_ENA   = 1U;
    NVIC_EnableIRQ(WULINS_IRQn);
```

3.2.2 Fix the issue of led false fault report by two-point calibration

3.2.2.1 issue description:

1) Fix LED false fault report due to LED PN Voltage after two-point calibration
2) When the calculated PN junction voltage is negative, it will exist in the form of a complement code, while the open circuit voltage detection is 4000U. Therefore, when the voltage is calculated to be negative, such as -1, it will be stored as 65535, which will cause false alarms

3.2.2.2 solution:

1) Store the calculated PN voltage as int type, and also store the short-circuit detection voltage as int type. When negative pressure occurs, there will be no error message

```c
int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_VOLT;
    int32_t sVolt;
    uint32_t uVolt;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sVolt = param->calibP.gain * (int32_t)sum + param->calibP.offset;
        if (sVolt <= 0){
            sVolt = 0;
        }
        uVolt = (uint32_t)sVolt;
        uVolt = uVolt >> MEASURE_GAIN_POS;
        volt = (int16_t)uVolt;
    }
    return volt;
}
```

```c
#if (LED_NUM <= 8U)
int16_t get_volt(AdcDatabuff_t *const param)
    int16_t volt = (int16_t)INVALID_VOLT;
    int32_t sVolt;
    uint32_t uVolt;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sVolt = param->calibP.gain * (int32_t)sum + param->calibP.offset;
        if(sVolt <=0){
            sVolt = -sVolt;
            uVolt = (uint32_t)sVolt;
            uVolt = uVolt >> MEASURE_GAIN_POS;
            volt = (int16_t)uVolt;
            volt = -volt;
        }else{
            uVolt = (uint32_t)sVolt;
            uVolt = uVolt >> MEASURE_GAIN_POS;
            volt = (int16_t)uVolt;
        }
    }
    return volt;
}
```

```c
/*
 *  Get Runtime Led PN voltage in mV
 *  @param [out] pnVoltR/pnVoltG/pnVoltB: 0-4000mV
 *  @return :none
 */
void MES_GetRunTimeLedPNVolt(LedNum_t ledIndex,uint16_t *pnVoltR, uint16_t *pnVoltG,uint16_t *pnVoltB
{
    *pnVoltR = (uint16_t)adcResult.vLedPN[ledIndex][0].target;
    *pnVoltG = (uint16_t)adcResult.vLedPN[ledIndex][1].target;
    *pnVoltB = (uint16_t)adcResult.vLedPN[ledIndex][2].target;
}
```

```c
#if (LED_NUM <= 8U)
/*
 *  Get Runtime Led PN voltage in mV
 *  @param [out] pnVoltR/pnVoltG/pnVoltB: 0-4000mV
 *  @return :none
 */
void MES_GetRunTimeLedPNVolt(LedNum_t ledIndex,int16_t *pnVoltR, int16_t *pnVoltG,int16_t *pnVoltB)
{
    *pnVoltR = adcResult.vLedPN[ledIndex][0].target;
    *pnVoltG = adcResult.vLedPN[ledIndex][1].target;
    *pnVoltB = adcResult.vLedPN[ledIndex][2].target;
}
```

```c
#define OVER_TEMPERATURE_HIGH          (130)   /*celsius degree */
#define OVER_TEMPERATURE_RELEASE       (OVER_TEMPERATURE_HIGH - 10)   /*celsius degree */

#define LED_PN_SHORT_VOLT                    (500U)   /* mV */
#define LED_PN_SHORT_OPEN                    (4000U)  /* mV */

#define INTENSITY_RATIO_TYPICAL              (1024U)                          /* 100% of ma
#define INTENSITY_RATIO_HIGH_TEMPERATURE     (INTENSITY_RATIO_TYPICAL*4U/5U)  /* 80% of ma
#define INTENSITY_RATIO_BATT_ISSUE           (0U)                             /* 0% of max
```

```c
#define OVER_TEMPERATURE_HIGH          (130)   /*celsius degree */
#define OVER_TEMPERATURE_RELEASE       (OVER_TEMPERATURE_HIGH - 10)   /*celsius degree */

#define LED_PN_SHORT_VOLT                    (500)   /* mV */
#define LED_PN_OPEN_VOLT                     (4000)  /* mV */

#define INTENSITY_RATIO_TYPICAL              (1024U)                          /* 100% of max i
#define INTENSITY_RATIO_HIGH_TEMPERATURE     (INTENSITY_RATIO_TYPICAL*4U/5U)  /* 80% of max int
#define INTENSITY_RATIO_BATT_ISSUE           (0U)                             /* 0% of max inte
```

```c
static void safetyHandle(void)
{
    /*  Led PN Voltage Protection Handling     */
    uint16_t pnVoltR,pnVoltG,pnVoltB;
    uint8_t pnState;
    for (uint8_t ledIndex = 0U; ledIndex < LED_NUM; ledIndex++){
        pnState = 0U;
        MES_GetRunTimeLedPNVolt((LedNum_t)ledIndex,&pnVoltR, &pnVoltG,&pnVoltB);
        if(PWM_LedIsOn(ledIndex) == 0U){
            pnVoltR = INVALID_VOLT;
            pnVoltG = INVALID_VOLT;
            pnVoltB = INVALID_VOLT;
        }
        if ( (pnVoltR != INVALID_VOLT) && (pnVoltG != INVALID_VOLT)  && (pnVoltB != INVALID_VOLT) ){
            if ( (pnVoltR <= LED_PN_SHORT_VOLT) || (pnVoltG <= LED_PN_SHORT_VOLT) || (pnVoltB <= LED_
                pnState = 1U;           /* short  */
            }
            if ( (pnVoltR >= LED_PN_SHORT_OPEN) || (pnVoltG >= LED_PN_SHORT_OPEN) || (pnVoltB >= LED_
                pnState |= (1U << 1);   /* open  */
            }
        }
        ledPNState[ledIndex] = (LedPNState_t)pnState;
    }
```

```c
static void safetyHandle(void)
{
    /*  Led PN Voltage Protection Handling     */
    int16_t pnVoltR,pnVoltG,pnVoltB;
    uint8_t pnState;
    for (uint8_t ledIndex = 0U; ledIndex < LED_NUM; ledIndex++){
        pnState = 0U;
        MES_GetRunTimeLedPNVolt((LedNum_t)ledIndex,&pnVoltR, &pnVoltG,&pnVoltB);
        if(PWM_LedIsOn(ledIndex) == 0U){
            pnVoltR = INVALID_VOLT;
            pnVoltG = INVALID_VOLT;
            pnVoltB = INVALID_VOLT;
        }
        if ( (pnVoltR != INVALID_VOLT) && (pnVoltG != INVALID_VOLT)  && (pnVoltB != INVALID_VOLT) ){
            if ( (pnVoltR <= LED_PN_SHORT_VOLT) || (pnVoltG <= LED_PN_SHORT_VOLT) || (pnVoltB <= LED_PN_S
                pnState = 1U;           /* short  */
            }
            if ( (pnVoltR >= LED_PN_OPEN_VOLT) || (pnVoltG >= LED_PN_OPEN_VOLT) || (pnVoltB >= LED_PN_OPE
                pnState |= (1U << 1);   /* open  */
            }
        }
        ledPNState[ledIndex] = (LedPNState_t)pnState;
    }
```

3.2.3 Fix the issue of system clock division mismatch

3.2.3.1 issue description:

1) The actual frequency division on the datasheet is the register value+1, while the frequency division value set in the code is the actual frequency division value=2 ^ register value

3.2.3.2 solution:

1) Modify the frequency division value macro definition in the code to be consistent with the datasheet

```c
#define CLOCK_DIV_1  (0U)
#define CLOCK_DIV_2  (1U)
#define CLOCK_DIV_4  (2U)
#define CLOCK_DIV_8  (3U)
```

```c
#ifndef CLOCK_DEVICE_H__
#define CLOCK_DIV_1  (0U)
#define CLOCK_DIV_2  (1U)
#define CLOCK_DIV_4  (3U)
#define CLOCK_DIV_8  (7U)
```

clock_device.c (工作树) 2, M

drivers > hal > src > clock_device.c > Clock_SystemMainClockInit(uint8_t)

```c
void Clock_SystemMainClockInit(uint8_t divider)
{
    CRGA_SFRS->SYSCLKCTRL.HFRCENA = 1U;
    while(CRGA_SFRS->SYSCLKCTRL.HFRCSTS == 0U){}
    /* Enable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0x0EU;
    SYSCTRLA_SFRS->HF_OSC_TRIM.TRIM_HF_RC = HWCFG_GetRCHFClockCalibValue();
    PMUAPRE5V_SFRS->CLK_CTRL.TRIM_LF_RC = HWCFG_GetRCLFClockCalibValue();
    CRGA_SFRS->LFCLKCTRL.CLKLFSEL = 1;
    /* Disable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0U;
    CRGA_SFRS->SYSCLKCTRL.SYSCLKSEL = (uint8_t)CLOCK_RC_16MHz;
    CRGA_SFRS->SYSCLKCTRL.DIVSYSCLK = (uint8_t)divider;
    /*  frequency spread    */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDEEP  = 7U;/*  3  */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDIV   = 15U;
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCENA   = 1U;

    systemClock = SYSTEM_CLOCK_HIGH_FREQ;
    systemClock = systemClock >> (uint8_t)divider;
}
```

```c
void Clock_SystemMainClockInit(uint8_t divider)
{
    CRGA_SFRS->SYSCLKCTRL.HFRCENA = 1U;
    while(CRGA_SFRS->SYSCLKCTRL.HFRCSTS == 0U){}
    /* Enable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0x0EU;
    SYSCTRLA_SFRS->HF_OSC_TRIM.TRIM_HF_RC = HWCFG_GetRCHFClockCalibValue();
    PMUAPRE5V_SFRS->CLK_CTRL.TRIM_LF_RC = HWCFG_GetRCLFClockCalibValue();
    CRGA_SFRS->LFCLKCTRL.CLKLFSEL = 1;
    /* Disable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0U;
    CRGA_SFRS->SYSCLKCTRL.SYSCLKSEL = (uint8_t)CLOCK_RC_16MHz;
    CRGA_SFRS->SYSCLKCTRL.DIVSYSCLK = (uint8_t)divider;
    /*  frequency spread    */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDEEP  = 7U;/*  3  */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDIV   = 15U;
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCENA   = 1U;

    systemClock = SYSTEM_CLOCK_HIGH_FREQ;
    systemClock = systemClock / (uint8_t)(divider + 1U);
}
```

4 V1.11

4.1 CHANGEVERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.11_lin_2.1.0_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.12_lin_2.1.0_colorlib_2.2.2

```
#define LIN_STACK_TYPE_ISO17987          LIN_STACK_TYPE_LIN2_2A

#define LIN_STACK_TYPE                   LIN_STACK_TYPE_LIN2_2A

#define SDK_APPL_VERSION                 (0x111U)
#define AUTO_ADDRESSING_EN               (1U)

/* **********************hardware config ***************************
#define MAIN_CPU_CLOCK          16U       /*MHz*/
```

```
#ifndef APP_CONFIG_H__

#define LIN_STACK_TYPE                   LIN_STACK_TYPE_LIN2_2A

#define SDK_APPL_VERSION                 (0x112U)
#define AUTO_ADDRESSING_EN               (1U)

/* **********************hardware config ***************************
#define MAIN_CPU_CLOCK          16U       /*MHz*/
```

4.2 BUG FIX

4.2.1 Update Temperature Calculation Method

```
#define MEASURE_GAIN_POS          12U
#define MEASURE_GAIN              4096 /*(1UL << MEASURE_GAIN_POS)*/

#define MEASURE_TEMP_GAIN_POS     14U
#define MEASURE_TEMP_GAIN         40960 /*16384*2.5 *2*/
```

```
#if (LED_NUM <= 8U)
#define MEASURE_GAIN_POS          12U
#define MEASURE_GAIN              4096 /*(1UL << MEASURE_GAIN_POS)*/

#define MEASURE_TEMP_GAIN_POS     14U
#define MEASURE_TEMP_GAIN         40960 /*16384*2.5 *2*/
#define MEASURE_ADC_GAIN_FACTOR   16913U /* 16384 / ADC_GAIN(31/32) = 16384 * 32/31 = 16913*/
```

```
int16_t get_chip_temp_volt(AdcDatabuff_t *const param)
{
    uint32_t bandgap_mV  = HWCFG_GetVBG();

    int16_t volt = (int16_t)INVALID_VOLT;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sum = (bandgap_mV*sum) >> 12U;/*1mV */
        volt = (int16_t)sum;
    }
    return volt;
}
```

```
#if (LED_NUM <= 8U)
int16_t get_chip_temp_volt(AdcDatabuff_t *const param)
{
    uint32_t bandgap_mV  = HWCFG_GetVBG();

    int16_t volt = (int16_t)INVALID_VOLT;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sum = (bandgap_mV*sum) >> 12U;/*1mV */
        volt = (int16_t)((uint32_t)((sum*MEASURE_ADC_GAIN_FACTOR) >> MEASURE_TEMP_GAIN_POS));
    }
    return volt;
}
```

```c
void measureParamInit(void)
{
    uint16_t version = HWCFG_GetFlashMapVersion();
    if (version == 0xFFFFU){
        dataIntegrityError = TRUE;
    }else{
        HWCFG_SFRS_t *param = (HWCFG_SFRS_t *)HWCFG_SFRS_ADDR;
        uint32_t crcCalculated = CRC32_GetRunTimeCRC32( param->payload + 4U ,508U);
        uint32_t crcFromFlash = HWCFG_SFRS->crc32;
        if (crcCalculated != crcFromFlash){
            dataIntegrityError = TRUE;
        }
    }

    uint32_t bandgap_mV  = HWCFG_GetVBG();
    uint16_t tSensorCode = HWCFG_GetADCCode_PTATLED();
    int32_t tSensor_mV;
    bandgap_mV = (bandgap_mV*tSensorCode) >> 12U;/* 0.5mV */
    tSensor_mV = (int32_t)bandgap_mV;
    adcResult.vChipTemp.startCode = INVALID_PARAM;
    /* T = volt /0.4 *- 271 */
    adcResult.tChip.gain        = MEASURE_TEMP_GAIN;
    adcResult.tChip.offset      = 30*(int32_t)(1UL << MEASURE_TEMP_GAIN_POS) - MEASURE_TEMP_GAIN*tSensor_mV;/* 30 de
```

```c
#if (LED_NUM <= 8U)
void measureParamInit(void)
{
    uint16_t version = HWCFG_GetFlashMapVersion();
    if (version == 0xFFFFU){
        dataIntegrityError = TRUE;
    }else{
        HWCFG_SFRS_t *param = (HWCFG_SFRS_t *)HWCFG_SFRS_ADDR;
        uint32_t crcCalculated = CRC32_GetRunTimeCRC32( param->payload + 4U ,508U);
        uint32_t crcFromFlash = HWCFG_SFRS->crc32;
        if (crcCalculated != crcFromFlash){
            dataIntegrityError = TRUE;
        }
    }

    uint32_t bandgap_mV  = HWCFG_GetVBG();
    uint16_t tSensorCode = HWCFG_GetADCCode_PTATLED();
    int32_t tSensor_mV;
    bandgap_mV = (bandgap_mV*tSensorCode) >> 12U;/* 0.5mV */
    tSensor_mV = (int32_t)((uint32_t)(bandgap_mV * MEASURE_ADC_GAIN_FACTOR) >> MEASURE_TEMP_GAIN_POS));
    adcResult.vChipTemp.startCode = INVALID_PARAM;
    /* T = volt /0.4 *- 271 */
    adcResult.tChip.gain        = MEASURE_TEMP_GAIN;
    adcResult.tChip.offset      = 25*(int32_t)(1UL << MEASURE_TEMP_GAIN_POS) - MEASURE_TEMP_GAIN*tSensor_mV;/* 25 degree
```

4.2.2 Add Handling Negative Voltage

```c
int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_VOLT;
    int32_t sVolt;
    uint32_t uVolt;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sVolt = param->calibP.gain * (int32_t)sum + param->calibP.offset;
        uVolt = (uint32_t)sVolt;
        uVolt = uVolt >> MEASURE_GAIN_POS;
        volt = (int16_t)uVolt;
    }
    return volt;
}
```

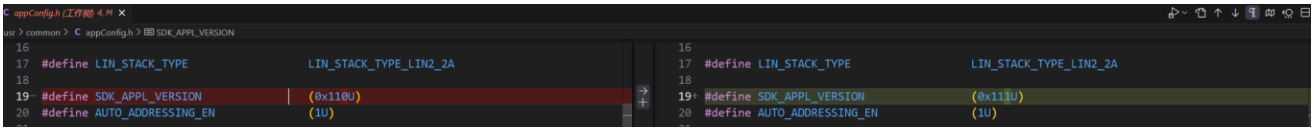
```c
#if (LED_NUM <= 8U)
int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_VOLT;
    int32_t sVolt;
    uint32_t uVolt;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sVolt = param->calibP.gain * (int32_t)sum + param->calibP.offset;
        if (sVolt <= 0){
            sVolt = 0;
        }
        uVolt = (uint32_t)sVolt;
        uVolt = uVolt >> MEASURE_GAIN_POS;
        volt = (int16_t)uVolt;
    }
    return volt;
}
```

5 V1.10

5.1 CHANGEVERSION

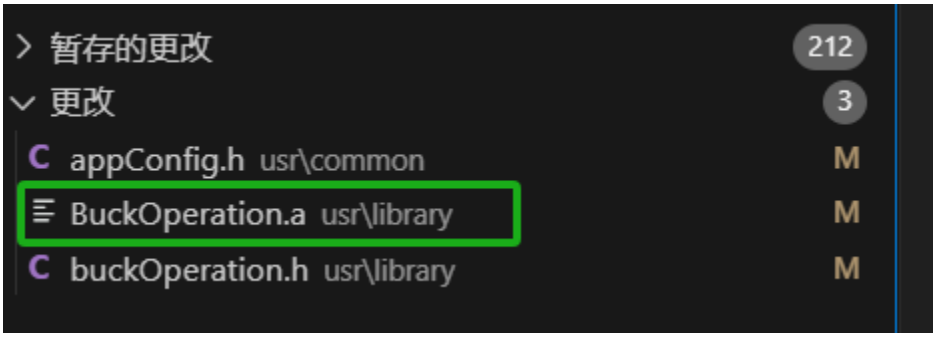
Form Version: libdev_iND83213A_CS01_App_B0.01.10_lin_2.1.0_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.11_lin_2.1.0_colorlib_2.2.2



5.2 BUG FIX

5.2.1 Fix the issue of Buck Set Freq Api Inaccurate settings



6 V1.09

6.1 Version Change

Form Version: libdev_iND83213A_CS01_App_B0.01.09_lin_2.1.0_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.10_lin_2.1.0_colorlib_2.2.2

appConfig.h (工作树) 2, M

usr > common > appConfig.h > SDK_APPL_VERSION

```
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_
18
19- #define SDK_APPL_VERSION                (0x109U)
20  #define AUTO_ADDRESSING_EN              (1U)
21
```

```
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2
18
19+ #define SDK_APPL_VERSION                (0x110U)
20  #define AUTO_ADDRESSING_EN              (1U)
21
```

6.2 Feature Update

6.2.1 Update PMU_EnterDeepSleepMode (void) API

pmu_device.c (工作树) 2, M

drivers > hal > src > pmu_device.c > PMU_EnterDeepSleepMode(void)

```
67
68  void PMU_EnterDeepSleepMode(void)
69  {
70      PMUAPRE5V_SFRS->CFG_ACCESS.CFG_ACCESS_KEY = 0x0BU;
71      BUCK_DisableBuck();                 /* disable buck
72      CRGA_SFRS->MODULERSTREQ     |= MODULE_SOFT_RESET_PW
73      LINS_SFRS->CTRL.SLEEP       = 0U;
74
75      /* disbale temperature sensor when sleep */
76      SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
77      SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;
```

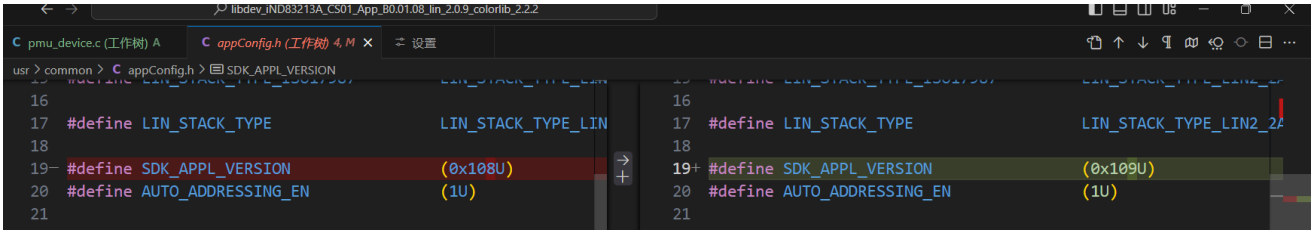
```
67
68  void PMU_EnterDeepSleepMode(void)
69  {
70      PMUAPRE5V_SFRS->CFG_ACCESS.CFG_ACCESS_KEY = 0x0BU;
71      BUCK_DisableBuck();                 /* disable buck*/
72      CRGA_SFRS->MODULERSTREQ     |= MODULE_SOFT_RESET_PWM | M
73      LINS_SFRS->CTRL.SLEEP       = 0U;
74
75+     PMUAPRE5V_SFRS->LINS.SWON               = 0U;
76+
77      /* disbale temperature sensor when sleep */
78      SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
79      SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;
```

7 V1.08

7.1 Version Change

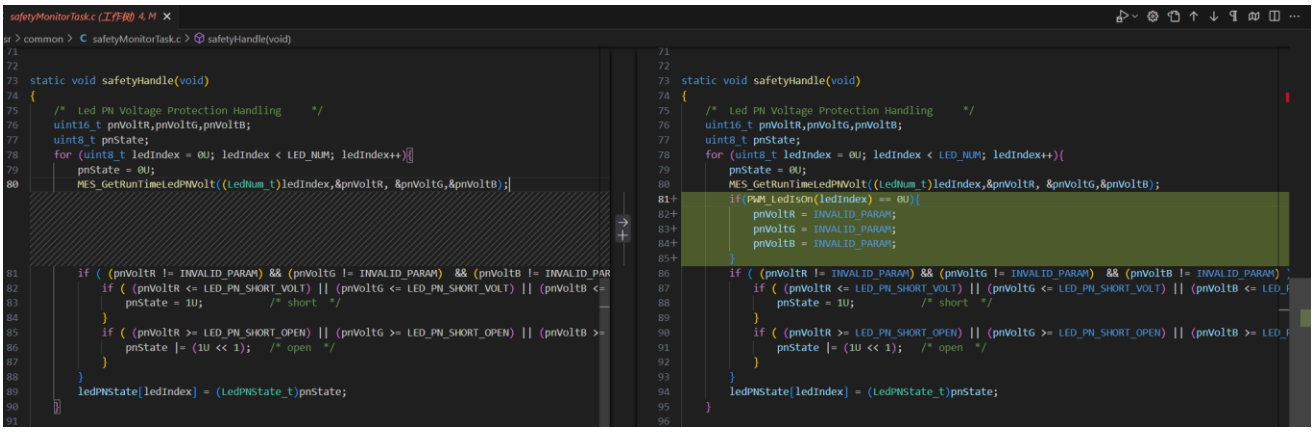
Form Version: libdev_iND83213A_CS01_App_B0.01.08_lin_2.0.9_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.09_lin_2.1.0_colorlib_2.2.2



7.2 Bug Fix

7.2.1 Fix the issue of reporting led error when it is turned off



7.3 NEW FEATURE

7.3.1 Add an linstack api for Once Force 3D response without 3C require.

```
/*
 *  @return TxStatus_t.
 */
TxStatus_t ls_tx_status(void);

/*
```

```
 *  frame response send status.
 *  @return TxStatus_t.
 */
TxStatus_t ls_tx_status(void);

/*
 *  workaround: require 3D response when no 3C request.
 *  @param [in] nad: the NAD required.
 *  @return none
 */
void ls_set_init_req_nad(uint8_t nad);
/*
```

This Feature is used for boot mode and app mode switch. And before use this function, user must call the below api and prepare data into LIN frame buff.

```
void LINS_TaskHandler(void)
{
  DiagRspInfo_t diagRsp={
    .sid       = 0,
    .type      = PDU_TYPE_SINGLE_PDU,
    .packLength = 6,
    .payload    = diagDataRspBuff,
  };
    switch(linsTaskState){
    case TASK_STATE_ACTIVE:
      break;
    case TASK_STATE_INIT:
      ls_register_services(LIN_PROTOCOL_LIN2_2A,UnconditionalCmdsTable, (l_u8)(sizeof(UnconditionalCmdsTable)/sizeof(LIN_Device_Frame_t)), DIAG_DATA_BUFF_SIZE,&linsFramesCall
      (void)ls_set_tp_timeout(N_AS, N_CR);
      (void)l_sys_init();

      ls_set_init_req_nad(0x05);
      ls_handle_diagnostic_response(&diagRsp);

      linsTaskState = TASK_STATE_ACTIVE;
      break;
    default:
      break;
    }
}
```

7.3.2 Add special functional NAD (0x7E) handler.

Ind Log

No	Source	Time Stamp	Delta	Command	Frame(Hex)/Delay(ms) NAD=0x7E测试	LEN	Data(String)
1	OUT	06:24:00:922	0.000	LIN_CMD	3C 7E 02 BA 0F FF FF FF FF	27	<~.....
2	OUT	06:24:00:944	0.022	LIN_CMD	3D	03	=.....
3	OUT	06:24:00:966	0.022	LIN_CMD	3D	03	=.....
4	OUT	06:24:00:988	0.022	LIN_CMD	3D	03	=.....
5	OUT	06:24:06:520	5.532	LIN_CMD	3C 7E 02 BA 0F FF FF FF FF	27	<~.....
6	OUT	06:24:06:542	0.022	LIN_CMD	3D	03	=.....
7	IN	06:24:06:556	0.014	LIN_CMD	3D 08 07 40 11 22 33 00 00	27	=..@."3..
8	OUT	06:24:06:564	0.008	LIN_CMD	3D	03	=.....
9	OUT	06:24:06:586	0.022	LIN_CMD	3D	03	=.....

main.c | linsNodeCfgIdentify.c | linStackTask.h | linslaveTask.c | linSlaveTask.h | taskManager.c

Diagnostic7EHandle(const NodeStruct_t *const)

```
70
71  static uint8_t eventTriggeredLedOn = FALSE;
72
73
74  void Diagnostic7EHandle(const NodeStruct_t * const frame)
75  {
76      NodeStruct_t testData = *frame;
77
78      DiagRspInfo_t diagRsp={
79        .sid      = 0,
80        .type     = PDU_TYPE_SINGLE_PDU,
81        .packLength = 6,
82        .payload  = diagDataRspBuff,
83      };
84      diagRsp.payload[0] = 0x11;
85      diagRsp.payload[1] = 0x22;
86      diagRsp.payload[2] = 0x33;
87      ls_handle_diagnostic_response(&diagRsp);
88  }
89
90
91  /*  LIN command handle callbacks declarations  */
92  static ls_LinsFramesCallback_t linsFramesCallback ={
93    UnconditionalSubscribedCmdsHandle,          /* received data from master */
94    UnconditionalPublishedCmdsISR,              /* send data to master, it's an interrupt function, please
95    UnconditionalPublishedCmdsTxFinishedISR,    /* send data to master finished, it's an interrupt functio
96    DiagnosticSubscribedCmdsHandle,             /* Diagnostic sigle PDU and MultiPDU received data from ma
97    Diagnostic7EHandle,                                 /* special functional NAD (0x7E) handler*/
98    DiagnosticSleepRequestHandle,               /* Diagnostic sleep request from master */
99    BusWakeUpRequestHandle,                     /* Get the bus wake up result this would take effects afte
100   NULL,                                       /* SAE J2602 -1 Configuration Messages */
101   NULL,                                       /*  5.7.2.5 DNN Based Broadcast Messages */
102 };
```

This Feature is used for special functional NAD (0x7E). And before use this function, user must call the below api and prepare data into LIN frame buff.

8 V1.07

8.1 Version Change

Form Version: libdev_iND83213A_CS01_App_B0.01.07_lin_2.0.9_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.08_lin_2.0.9_colorlib_2.2.2

appConfig.h > ...

```
#define LIN_STACK_TYPE_LIN2_2A          (0U)
#define LIN_STACK_TYPE_SAEJ2602         (1U)
#define LIN_STACK_TYPE_PRIVATE          (2U)
#define LIN_STACK_TYPE_ISO17987         LIN_STACK_TYPE_LIN2_2A

#define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A

-#define SDK_APPL_VERSION                (0x107U)
#define AUTO_ADDRESSING_EN              (1U)
```

```
#define LIN_STACK_TYPE_LIN2_2A          (0U)
#define LIN_STACK_TYPE_SAEJ2602         (1U)
#define LIN_STACK_TYPE_PRIVATE          (2U)
#define LIN_STACK_TYPE_ISO17987         LIN_STACK_TYPE_LIN2_2A

#define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A

+#define SDK_APPL_VERSION                (0x108U)
#define AUTO_ADDRESSING_EN              (1U)
```

8.2 BUG Fix

8.2.1 Resolve the issue of abnormal sleep mode branches

pmu_device.c > ...

```
/**
 * @copyright 2017 Indie Semiconductor.
 *
 * This file is proprietary to Indie Semiconductor.
 * All rights reserved. Reproduction or distribution, in whole
 * or in part, is forbidden except by express written permission
 * of Indie Semiconductor.
 *
 * @file pmu_device.c
 * @Author: Jack.Pan
 * @E-mail:jack.pan@indiemicro.com
 * @Date: 2020/03/18
 */

#include <stddef.h>
#include <pmu_device.h>
#include <errno.h>
#include <isrfuncs.h>
#include <gpio_device.h>
#include <lin_device.h>
#include <wica_sfr.h>
#include <pwm_device.h>
#include <buck_device.h>
#include <wdt_device.h>
```

```
/**
 * @copyright 2017 Indie Semiconductor.
 *
 * This file is proprietary to Indie Semiconductor.
 * All rights reserved. Reproduction or distribution, in whole
 * or in part, is forbidden except by express written permission
 * of Indie Semiconductor.
 *
 * @file pmu_device.c
 * @Author: Jack.Pan
 * @E-mail:jack.pan@indiemicro.com
 * @Date: 2020/03/18
 */

+#include <appConfig.h>
#include <stddef.h>
#include <pmu_device.h>
#include <errno.h>
#include <isrfuncs.h>
#include <gpio_device.h>
#include <lin_device.h>
#include <wica_sfr.h>
#include <pwm_device.h>
#include <buck_device.h>
#include <wdt_device.h>
```

pmu_device.c > ...

```
{
  }
}

void PMU_EnterDeepSleepMode(void)
{
    PMUAPRESV_SFRS->CFG_ACCESS.CFG_ACCESS_KEY = 0x0BU;
    BUCK_DisableBuck();            /* disable buck*/
    CRGA_SFRS->MODULERSTREQ      |= MODULE_SOFT_RESET_PWM | MODULE_SOFT_RESET_ADC | MODULE_SOFT_RESET_BUCK;
    LINS_SFRS->CTRL.SLEEP        = 0U;

    /* disbale temperature sensor when sleep */
    SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
    SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;

    PMUA_SFRS->VBATCTRL.BAT_UV_EN = 0U;
    PMUA_SFRS->VBATCTRL.BAT_OV_EN = 0U;

    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRESV_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */
    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.SLEEP = 1U;)  */
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRESV_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;
#if (LIN_STACK_TYPE == LIN_STACK_TYPE_PRIVATE)
    /* Set lin to sleep mode */
    if (LINS_SFRS->CTRL.SLEEP == 1U){
        LINS_SFRS->CTRL.SLEEP = 0U;
    }
    LINS_SFRS->CTRL.SLEEP = 1U;
    /*  Enter sleep mode  */
    PMUA_SFRS->CTRL.HIBERNATE = 1U;
#else
    NVIC_EnableIRQ(Lullaby_IRQn);              /*  Enable  Lullaby interrupt*/
    EVTHOLD_SFRS->HOLD = 1U;                   /*  Trigger ISR then entering sleep in ISR */
#endif
```

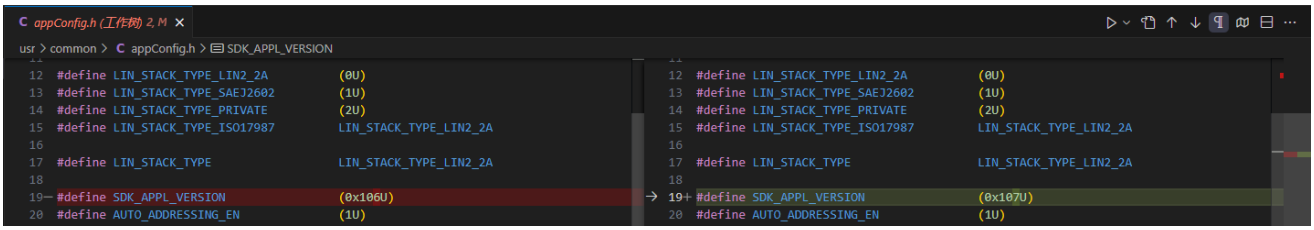
```
void PMU_WakeTimerInit(PMU_WAKEUP_TIMEER_MODE_t mode, PMU_WAKEUP_TIMEER_Interval_t interval)
  }
}

void PMU_EnterDeepSleepMode(void)
{
    PMUAPRESV_SFRS->CFG_ACCESS.CFG_ACCESS_KEY = 0x0BU;
    BUCK_DisableBuck();            /* disable buck*/
    CRGA_SFRS->MODULERSTREQ      |= MODULE_SOFT_RESET_PWM | MODULE_SOFT_RESET_ADC | MODULE_SOFT_RESET_BUCK;
    LINS_SFRS->CTRL.SLEEP        = 0U;

    /* disbale temperature sensor when sleep */
    SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
    SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;

    PMUA_SFRS->VBATCTRL.BAT_UV_EN = 0U;
    PMUA_SFRS->VBATCTRL.BAT_OV_EN = 0U;

    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRESV_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */
    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.SLEEP = 1U;)  */
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRESV_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;
#if (LIN_STACK_TYPE == LIN_STACK_TYPE_PRIVATE)
    /* Set lin to sleep mode */
    if (LINS_SFRS->CTRL.SLEEP == 1U){
        LINS_SFRS->CTRL.SLEEP = 0U;
    }
    LINS_SFRS->CTRL.SLEEP = 1U;
    /*  Enter sleep mode  */
    PMUA_SFRS->CTRL.HIBERNATE = 1U;
#else
    NVIC_EnableIRQ(Lullaby_IRQn);              /*  Enable  Lullaby interrupt*/
    EVTHOLD_SFRS->HOLD = 1U;                   /*  Trigger ISR then entering sleep in ISR */
#endif
```

9 V1.06

9.1 CHANGEVERSION

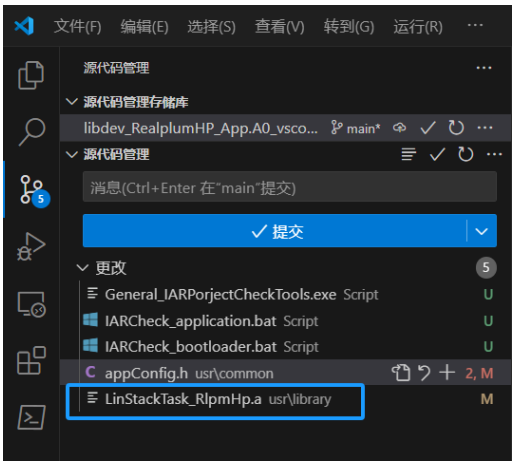
Form Version: libdev_iND83213A_CS01_App_B0.01.06_lin_2.0.8_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.07_lin_2.0.9_colorlib_2.2.2



9.2 LIN STACK UPDATE

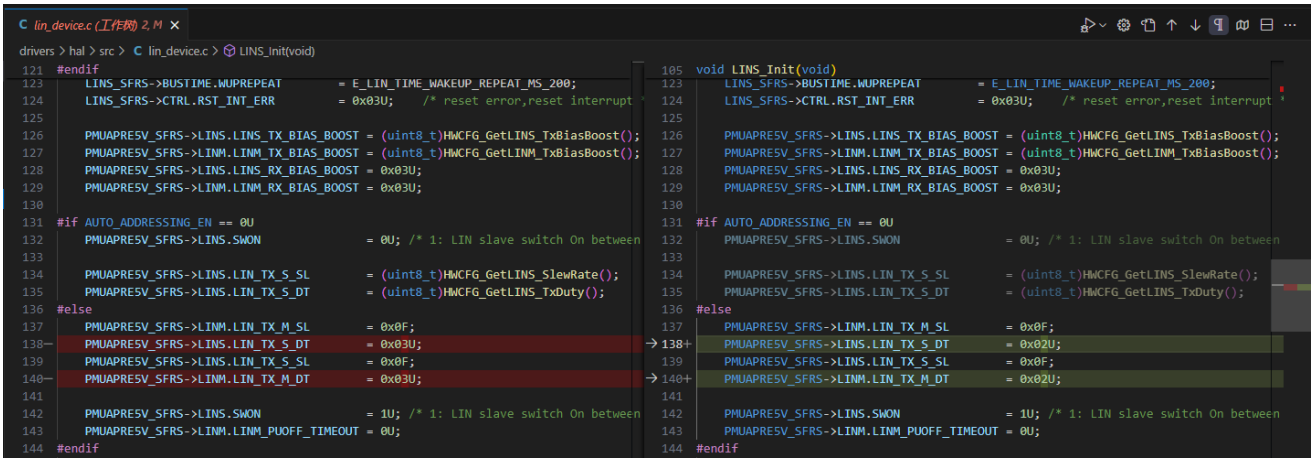
9.2.1 Update error report logic @J2602 Mode



9.3 BUG FIXED

9.3.1 Update LIN device Params for improve LIN_AA performance

New configuration can improve LIN Duty after multiple nodes.



```
#endif
    LINS_SFRS->BUSTIME.WUPREPEAT          = E_LIN_TIME_WAKEUP_REPEAT_MS_200;
    LINS_SFRS->CTRL.RST_INT_ERR           = 0x03U;    /* reset error,reset interrupt

    PMUAPRE5V_SFRS->LINS.LINS_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINS_TxBiasBoost();
    PMUAPRE5V_SFRS->LINM.LINM_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINM_TxBiasBoost();
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0x03U;
    PMUAPRE5V_SFRS->LINM.LINM_RX_BIAS_BOOST = 0x03U;

#if AUTO_ADDRESSING_EN == 0U
    PMUAPRE5V_SFRS->LINS.SWON               = 0U; /* 1: LIN slave switch On between

    PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = (uint8_t)HWCFG_GetLINS_SlewRate();
    PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = (uint8_t)HWCFG_GetLINS_TxDuty();
#else
    PMUAPRE5V_SFRS->LINM.LIN_TX_M_SL        = 0x0F;
    PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = 0x03U;
    PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = 0x0F;
    PMUAPRE5V_SFRS->LINM.LIN_TX_M_DT        = 0x03U;

    PMUAPRE5V_SFRS->LINS.SWON               = 1U; /* 1: LIN slave switch On between
    PMUAPRE5V_SFRS->LINM.LINM_PUOFF_TIMEOUT = 0U;
#endif
```

```
void LINS_Init(void)
    LINS_SFRS->BUSTIME.WUPREPEAT          = E_LIN_TIME_WAKEUP_REPEAT_MS_200;
    LINS_SFRS->CTRL.RST_INT_ERR           = 0x03U;    /* reset error,reset interrupt

    PMUAPRE5V_SFRS->LINS.LINS_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINS_TxBiasBoost();
    PMUAPRE5V_SFRS->LINM.LINM_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINM_TxBiasBoost();
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0x03U;
    PMUAPRE5V_SFRS->LINM.LINM_RX_BIAS_BOOST = 0x03U;

#if AUTO_ADDRESSING_EN == 0U
    PMUAPRE5V_SFRS->LINS.SWON               = 0U; /* 1: LIN slave switch On between

    PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = (uint8_t)HWCFG_GetLINS_SlewRate();
    PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = (uint8_t)HWCFG_GetLINS_TxDuty();
#else
    PMUAPRE5V_SFRS->LINM.LIN_TX_M_SL        = 0x0F;
    PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = 0x02U;
    PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = 0x0F;
    PMUAPRE5V_SFRS->LINM.LIN_TX_M_DT        = 0x02U;

    PMUAPRE5V_SFRS->LINS.SWON               = 1U; /* 1: LIN slave switch On between
    PMUAPRE5V_SFRS->LINM.LINM_PUOFF_TIMEOUT = 0U;
#endif
```

TestData: 220pF @LININ and 220pF @LINOUT VBAT=13.5V. 16 EVK. condition.

Num of 213ACS01EVK = 16;VBAT = 13V5					
Conditon	FID	SamplePoint：Node1 slave data field		SamplePoint：Nodex master sync field	
		duty(us)	duty(%)	duty(us)	duty(%)
VBAT=13.5V LINS_RX BIAS_BOOST =3 LINM_RX_BIAS_BOOST = 3 LINS_SLEWRATE = 0xF LINM_SLEWRATE = 0xF LINS_TX_S_DT = 2 LINM_TX_S_DT = 2	0x10	52.41	50.39	50.92	48.96
	0xF	52.82	50.79	50.97	49.01
	0xE	52.4	50.38	50.89	48.93
	0xD	52.4	50.38	50.81	48.86
	0xC	52.79	50.76	50.97	49.01
	0xB	53.24	51.19	51.01	49.05
	0xA	52.92	50.88	50.92	48.96
	0x9	53.25	51.20	50.88	48.92
	0x8	53.42	51.37	50.71	48.76
	0x7	52.4	50.38	50.92	48.96
	0x6	53.19	51.14	50.76	48.81
	0x5	53.19	51.14	50.81	48.86
	0x4	53.19	51.14	51.18	49.21
	0x3	52.98	50.94	50.92	48.96
	0x2	52.5	50.48	51.03	49.07
	0x1	52.5	50.48	51.03	49.07

9.3.2 Fix pwm device driver API error

pwm_device.c (工作树) 2, M

drivers > hal > src > pwm_device.c > PWM_Handler(void)

```c
static uint16_t currPWM[24U];
static PwmIsrCallback_t pwmReloadCallback = NULL;

void PWM_Handler(void)
{
    if (pwmReloadCallback == NULL){
        pwmReloadCallback();
    }
}
```

```c
static uint16_t currPWM[24U];
static PwmIsrCallback_t pwmReloadCallback = NULL;

void PWM_Handler(void)
{
    if (pwmReloadCallback != NULL){
        pwmReloadCallback();
    }
}
```

pwm_device.c (工作树) 2, M

drivers > hal > src > pwm_device.c > PWM_Handler(void)

```c
void PWM_RegisterPWMReloadIRQ(PwmIsrCallback_t callback)
{
    pwmReloadCallback = callback;
    PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
    PWM_SFRS->INTPWM.IRQ.PERIOD = 1U;
    PWM_SFRS->UPDATE = 1U;
    NVIC_EnableIRQ(PWM_IRQn);
}

/**
 * @brief The instance of PMW device
 */
void PWM_UnRegisterPWMReloadIRQ(void)
{
    pwmReloadCallback = NULL;
    PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
    PWM_SFRS->INTPWM.IRQ.PERIOD   = 0U;
    PWM_SFRS->UPDATE = 1U;
    NVIC_DisableIRQ(PWM_IRQn);
}
```

```c
void PWM_RegisterPWMReloadIRQ(PwmIsrCallback_t callback)
{
    pwmReloadCallback = callback;
    PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
    PWM_SFRS->INTPWM.ENABLE.PERIOD = 1U;
    PWM_SFRS->UPDATE = 1U;
    NVIC_EnableIRQ(PWM_IRQn);
}

/**
 * @brief The instance of PMW device
 */
void PWM_UnRegisterPWMReloadIRQ(void)
{
    pwmReloadCallback = NULL;
    PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
    PWM_SFRS->INTPWM.ENABLE.PERIOD   = 0U;
    PWM_SFRS->UPDATE = 1U;
    NVIC_DisableIRQ(PWM_IRQn);
}
```

10V1.05

10.1CHANGEVERSION

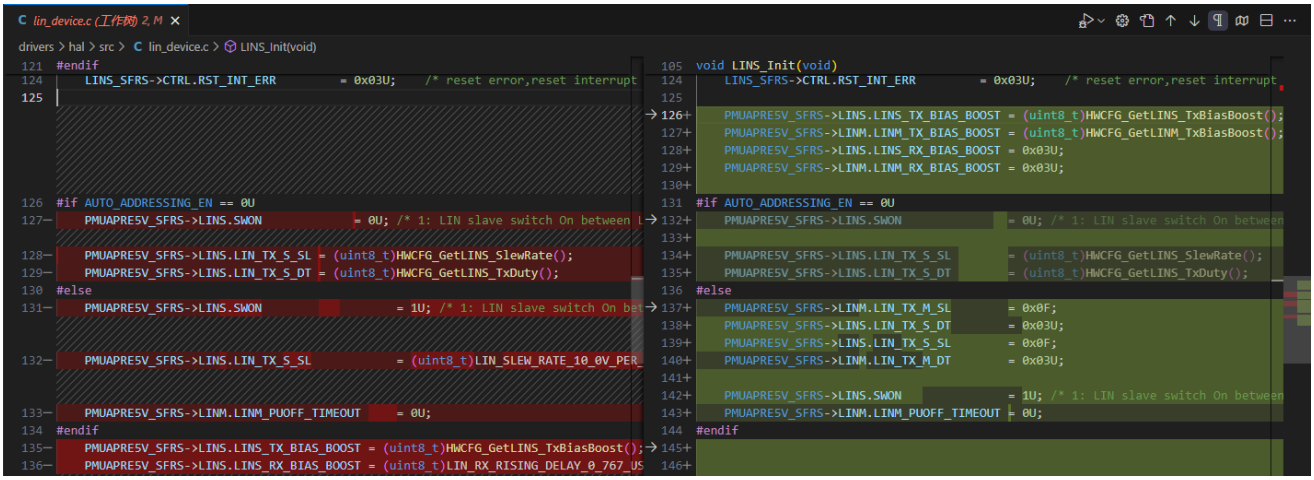
Form Version: libdev_iND83213A_CS01_App_B0.01.05_lin_2.0.8_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.06_lin_2.0.8_colorlib_2.2.2

10.2BUG FIXED

10.2.1 Update LIN device Params for improve LIN_AA performance

New configuration can improve LIN Duty after multiple nodes.



10.2.2 Fix sleep issue @Private_linsavetask mode

pmu_device.c (工作树) 2, M

drivers > hal > src > pmu_device.c > PMU_EnterDeepSleepMode(void)

```c
{
    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRE5V_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */
    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.S
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;

    NVIC_EnableIRQ(Lullaby_IRQn);                /*  Enable  Lullaby interrupt*/
    EVTHOLD_SFRS->HOLD = 1U;                     /*  Trigger ISR then entering s

    /* Waiting to enter sleep mode */
    for(;;){
    }
}
```

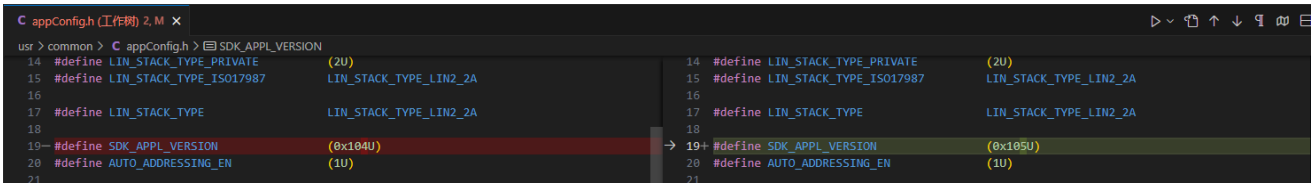
```c
void PMU_EnterDeepSleepMode(void)
    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRE5V_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */
    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.S
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;
+#if (LIN_STACK_TYPE == LIN_STACK_TYPE_PRIVATE)
+    /* Set lin to sleep mode */
+    if (LINS_SFRS->CTRL.SLEEP == 1U){
+        LINS_SFRS->CTRL.SLEEP = 0U;
+    }
+    LINS_SFRS->CTRL.SLEEP = 1U;
+    /*  Enter sleep mode  */
+    PMUA_SFRS->CTRL.HIBERNATE = 1U;
+#else
    NVIC_EnableIRQ(Lullaby_IRQn);                /*  Enable  Lullaby interrupt*/
    EVTHOLD_SFRS->HOLD = 1U;                     /*  Trigger ISR then entering s
+#endif
    /* Waiting to enter sleep mode */
    for(;;){
    }
}
```

11 V1.04

11.1 CHANGE VERSION

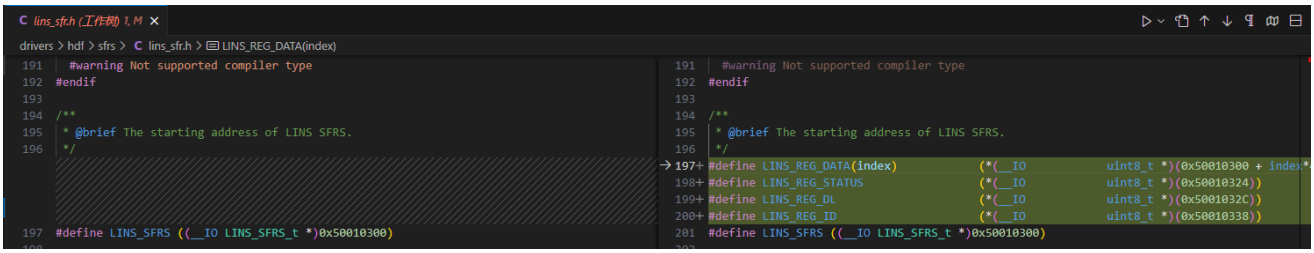
Form Version: libdev_iND83213A_CS01_App_B0.01.04_lin_2.0.8_colorlib_2.2.2

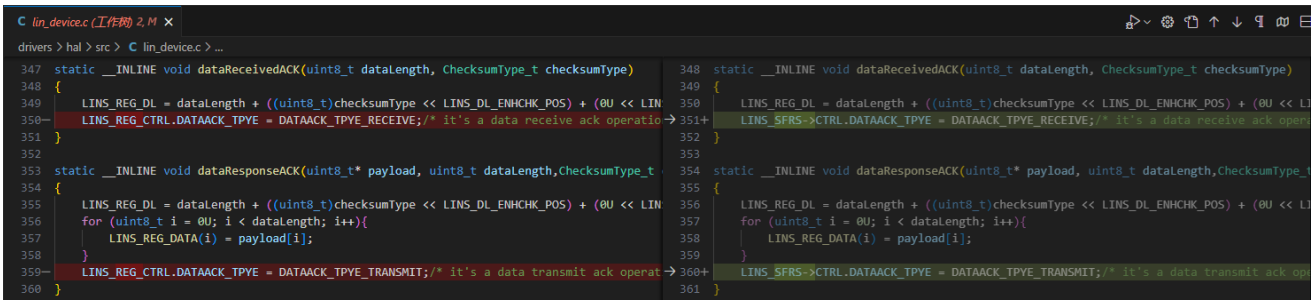
Update Version: libdev_iND83213A_CS01_App_B0.01.05_lin_2.0.8_colorlib_2.2.2



11.2 BUG FIXED

11.2.1 Proting linslaveTask_Private.c





12 V1.03

12.1 Change Version

Form Version: libdev_iND83213A_CS01_App_B0.01.03_lin_2.0.8_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.04_lin_2.0.8_colorlib_2.2.2

appConfig.h (工作树) 2, M

usr > common > appConfig.h > SDK_APPL_VERSION

```
13  #define LIN_STACK_TYPE_SAEJ2602          (1U)
14  #define LIN_STACK_TYPE_PRIVATE           (2U)
15  #define LIN_STACK_TYPE_ISO17987          LIN_STACK_TYPE_LIN2_2A
16
17  #define LIN_STACK_TYPE                   LIN_STACK_TYPE_LIN2_2A
18
19- #define SDK_APPL_VERSION                 (0x103U)
20  #define AUTO_ADDRESSING_EN               (1U)
21
```

```
13  #define LIN_STACK_TYPE_SAEJ2602          (1U)
14  #define LIN_STACK_TYPE_PRIVATE           (2U)
15  #define LIN_STACK_TYPE_ISO17987          LIN_STACK_TYPE_LIN2_2A
16
17  #define LIN_STACK_TYPE                   LIN_STACK_TYPE_LIN2_2A
18
19+ #define SDK_APPL_VERSION                 (0x104U)
20  #define AUTO_ADDRESSING_EN               (1U)
21
```

12.2 Default Status Modify

12.2.1 Default Params Modify For SNPD

adc_device.c (工作树) 3, M

drivers > hal > src > adc_device.c > ADC_GeneralInit(void)

```
59  #else
60      #error MAIN_CPU_CLOCK MUST BE at 16MHz system colck!
61  #endif
62
63      SAR_CTRL_SFRS->SAR_CTRL.TRIGSEL          = (uint8_t)ADC_TRIGGER_SOURCE_SOFT
64      SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY         = 4U;        /* 0.25us/count @4MH
65-     SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY_TS      = 4U;
66
```

```
59  #else
60      #error MAIN_CPU_CLOCK MUST BE at 16MHz system colck!
61  #endif
62
63      SAR_CTRL_SFRS->SAR_CTRL.TRIGSEL          = (uint8_t)ADC_TRIGGER_SOURCE_SO
64      SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY         = 4U;        /* 0.25us/count @4
65+     SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY_TS      = 32U;
66
```

adc_device_2_time_division_multiplex.c (工作树) 2, M

drivers > hal > src > adc_device_2_time_division_multiplex.c > ADC_GeneralInit(void)

```
11  {
12  #if SYS_MAIN_CLOCK_DIV == CLOCK_DIV_1
13      SAR_CTRL_SFRS->SAR_CFG2.SAR_CLK_DIV      = 3U;
14  #else
15      #error MAIN_CPU_CLOCK MUST BE at 16MHz system colck!
16  #endif
17
18      SAR_CTRL_SFRS->SAR_CTRL.TRIGSEL          = (uint8_t)ADC_TRIGGER_SOURCE_SOFT
19      SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY         = 4U;        /* 0.25us/count @4MH
20-     SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY_TS      = 4U;
```

```
11  {
12  #if SYS_MAIN_CLOCK_DIV == CLOCK_DIV_1
13      SAR_CTRL_SFRS->SAR_CFG2.SAR_CLK_DIV      = 3U;
14  #else
15      #error MAIN_CPU_CLOCK MUST BE at 16MHz system colck!
16  #endif
17
18      SAR_CTRL_SFRS->SAR_CTRL.TRIGSEL          = (uint8_t)ADC_TRIGGER_SOURCE_SOF
19      SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY         = 4U;        /* 0.25us/count @4
20+     SAR_CTRL_SFRS->SAR_CFG2.TRIG_DLY_TS      = 32U;
```

13 V1.02

13.1 CHANGE VERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.02_lin_2.0.8_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.03_lin_2.0.8_colorlib_2.2.2

```
appConfig.h (工作树) 2, M
usr > common > appConfig.h > SDK_APPL_VERSION
16
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A
18
19- #define SDK_APPL_VERSION                (0x102U)
20  #define AUTO_ADDRESSING_EN              (1U)
21
```

```
16
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A
18
19+ #define SDK_APPL_VERSION                (0x103U)
20  #define AUTO_ADDRESSING_EN              (1U)
21
```

13.2 BUG FIXED

13.2.1 Modify code position

```
adc_device.c (工作树) 3, M
drivers > hal > src > adc_device.c > ADC_Handler(void)
26  void ADC_Handler(void)
27  {
28      if (adcCallback !=NULL){
29          adcResult[0] = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
30          adcResult[1] = SAR_CTRL_SFRS->ADC_DATA0.DATA1;
31          adcResult[2] = SAR_CTRL_SFRS->ADC_DATA1.DATA2;
32          adcResult[3] = SAR_CTRL_SFRS->ADC_DATA1.DATA3;
33          adcCallback(adcMeasParamm, adcResult);
34      }
35
36      if((adcResult[0]&0x800U) == 0x800U){
37        adcResult[0] = 0;
38      }
39      if((adcResult[1]&0x800U) == 0x800U){
40        adcResult[1] = 0;
41      }
42      if((adcResult[2]&0x800U) == 0x800U){
43        adcResult[2] = 0;
44      }
45      if((adcResult[3]&0x800U) == 0x800U){
46        adcResult[3] = 0;
47      }
48      SAR_CTRL_SFRS->SAR_INT.CLEAR.INT_SAR_DONE_CLR = 1U;
49      SAR_CTRL_SFRS->SAR_CTRL.SAR_ENA_REQ = 0U;
50  }
```

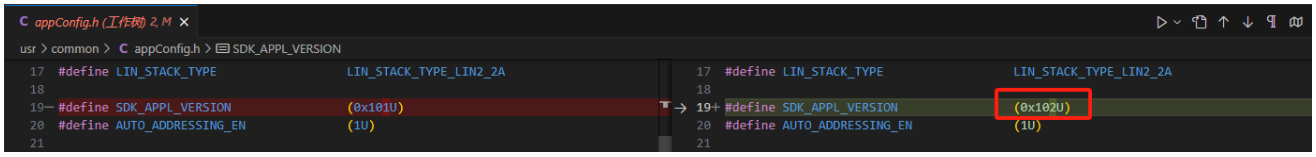
```
26  void ADC_Handler(void)
27  {
28      if (adcCallback !=NULL){
29          adcResult[0] = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
30          adcResult[1] = SAR_CTRL_SFRS->ADC_DATA0.DATA1;
31          adcResult[2] = SAR_CTRL_SFRS->ADC_DATA1.DATA2;
32          adcResult[3] = SAR_CTRL_SFRS->ADC_DATA1.DATA3;
33
34          if((adcResult[0]&0x800U) == 0x800U){
35            adcResult[0] = 0;
36          }
37          if((adcResult[1]&0x800U) == 0x800U){
38            adcResult[1] = 0;
39          }
40          if((adcResult[2]&0x800U) == 0x800U){
41            adcResult[2] = 0;
42          }
43          if((adcResult[3]&0x800U) == 0x800U){
44            adcResult[3] = 0;
45          }
46+         adcCallback(adcMeasParamm, adcResult);
47+     }
48+
49+
50+
51      SAR_CTRL_SFRS->SAR_INT.CLEAR.INT_SAR_DONE_CLR = 1U;
52      SAR_CTRL_SFRS->SAR_CTRL.SAR_ENA_REQ = 0U;
53  }
```

14 V1.01

14.1 CHANGE VERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.01_lin_2.0.8_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.02_lin_2.0.8_colorlib_2.2.2



14.2 BUG FIXED

14.2.1 None

14.3 New Feature

14.3.1 Add ADC Code Filter

Force negative adc code to be 0.

adc_device.c (工作树) 3, M

drivers > hal > src > adc_device.c > ADC_Handler(void)

```c
void ADC_Handler(void)
{
    if (adcCallback !=NULL){
        adcResult[0] = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        adcResult[1] = SAR_CTRL_SFRS->ADC_DATA0.DATA1;
        adcResult[2] = SAR_CTRL_SFRS->ADC_DATA1.DATA2;
        adcResult[3] = SAR_CTRL_SFRS->ADC_DATA1.DATA3;
        adcCallback(adcMeasParamm, adcResult);
    }
    SAR_CTRL_SFRS->SAR_INT.CLEAR.INT_SAR_DONE_CLR = 1U;
    SAR_CTRL_SFRS->SAR_CTRL.SAR_ENA_REQ = 0U;
}
```

```c
void ADC_Handler(void)
{
    if (adcCallback !=NULL){
        adcResult[0] = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        adcResult[1] = SAR_CTRL_SFRS->ADC_DATA0.DATA1;
        adcResult[2] = SAR_CTRL_SFRS->ADC_DATA1.DATA2;
        adcResult[3] = SAR_CTRL_SFRS->ADC_DATA1.DATA3;
        adcCallback(adcMeasParamm, adcResult);
    }

    if((adcResult[0]&0x800U) == 0x800U){
      adcResult[0] = 0;
    }
    if((adcResult[1]&0x800U) == 0x800U){
      adcResult[1] = 0;
    }
    if((adcResult[2]&0x800U) == 0x800U){
      adcResult[2] = 0;
    }
    if((adcResult[3]&0x800U) == 0x800U){
      adcResult[3] = 0;
    }
    SAR_CTRL_SFRS->SAR_INT.CLEAR.INT_SAR_DONE_CLR = 1U;
    SAR_CTRL_SFRS->SAR_CTRL.SAR_ENA_REQ = 0U;
}
```

pwm_device_2_time_division_multiplex.c (工作树) 2, M

drivers > hal > src > pwm_device_2_time_division_multiplex.c > ADC_Handler(void)

```c
        adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        vLedPNCode[ledNo][ledChannel]   = adcCode;
    }
    adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA1; vBattCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA2; vBuckCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA3; vTempCode = adcCode;
    if (++ledNo >= LED_NUM){
        ledNo = 0U;
        if (++ledChannel >= 3U){
            ledChannel = 0U;
        }
    }
```

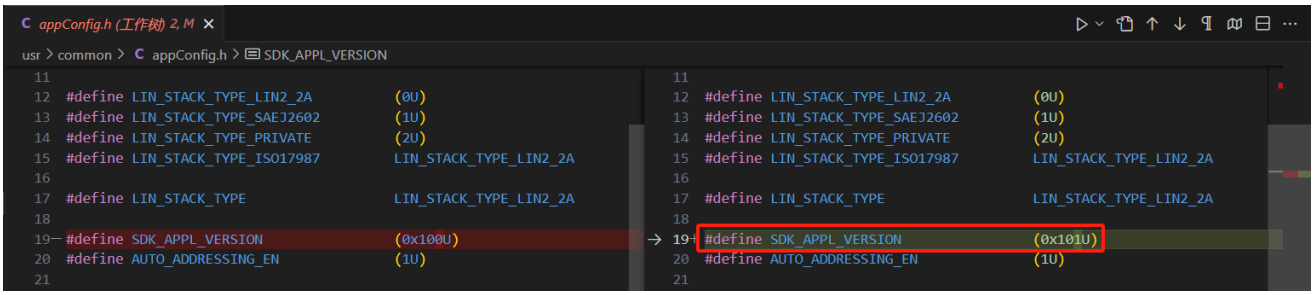
```c
        adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        if(adcCode&0x800U) == 0x800U){
          adcCode = 0;
        }
        vLedPNCode[ledNo][ledChannel]   = adcCode;
    }
    adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA1; vBattCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA2; vBuckCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA3; vTempCode = adcCode;

    if((vBattCode&0x800U) == 0x800U){
      vBattCode = 0;
    }
    if((vBuckCode&0x800U) == 0x800U){
      vBuckCode = 0;
    }
    if((vTempCode&0x800U) == 0x800U){
      vTempCode = 0;
    }
    if (++ledNo >= LED_NUM){
        ledNo = 0U;
        if (++ledChannel >= 3U){
            ledChannel = 0U;
        }
    }
```

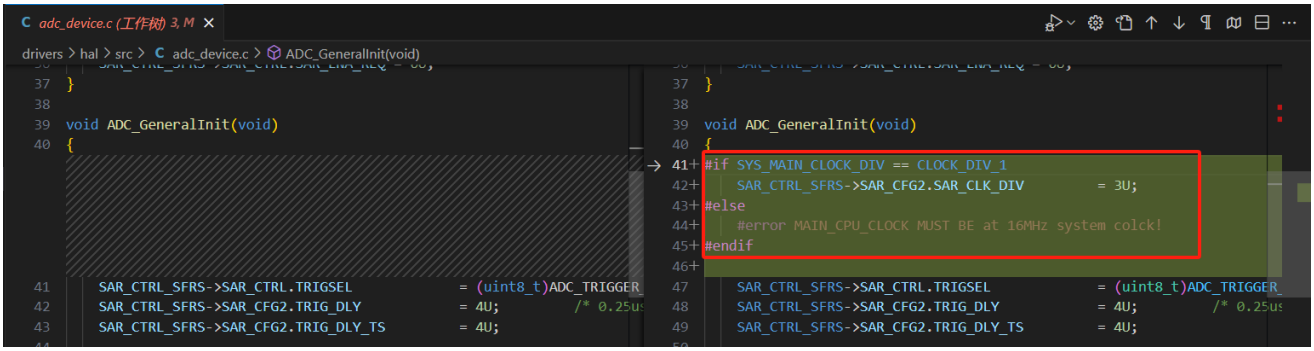
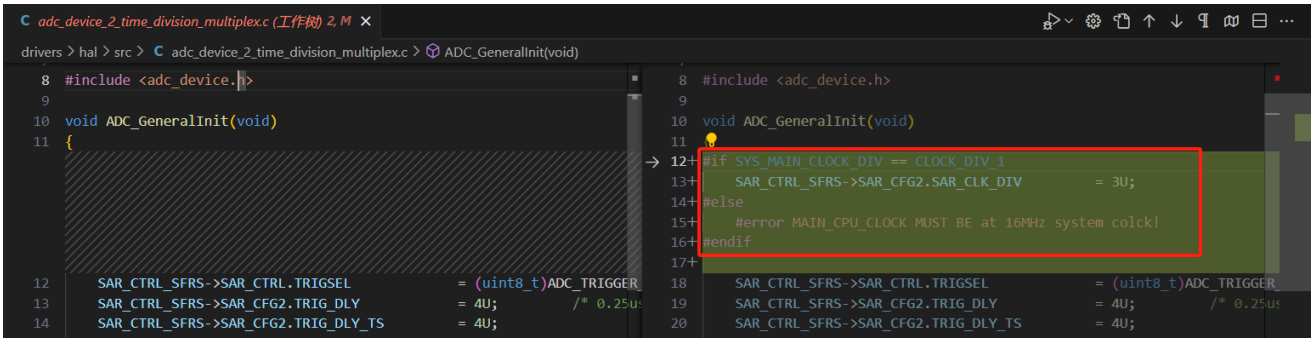
15 V1.00

15.1 CHANGE VERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.00_lin_2.0.8_colorlib_2.2.2

Update Version: libdev_iND83213A_CS01_App_B0.01.01_lin_2.0.8_colorlib_2.2.2



15.2 BUG FIXED

15.2.1 Make ADC clock source to be 4MHz